

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-aa31b716c99a7fb0b.jsonl`  
- SHA-256: `158bd1d1ff4b1cbcd9d087d5c055420804b13929a0f9ba3f38aef40ddd7b64e4`  
- Source modified: `2026-07-06T06:14:56+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:08:32.724Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.__apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact code at the cited lines, trace the real control flow, and if useful run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return REFUTED if the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff). Return PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension schema_sql):

title: Backfill fingerprint-collapse can drop a matched=1 video in favor of a matched=0 duplicate
severity(claimed): low
file:line: src/tg_video_filter/db.py:539
summary: content_items backfill is INSERT OR IGNORE under UNIQUE(account_id, dedup_key); two

distinct legacy videos sharing (size_bytes,duration_s,height,width) with NULL document_id collapse to one tg:fp: key, and OR IGNORE keeps whichever has the lower rowid, which may be the matched=0 row, silently discarding the matched=1 signal of the dropped row.

failure_scenario: v1 had NO DB-level fingerprint uniqueness (only UNIQUE(user_id,channel_id,message_id) and UNIQUE(user_id,document_id) WHERE document_id IS NOT NULL). Two distinct videos in different channels with identical (dur=60,w=640,h=480,size=1000), NULL document_id, first matched=0 (rowid 1) and second matched=1 (rowid 2) can coexist in a v1 DB (e.g. inserted via a fingerprint-lookup race in telethon_client.py:207-222, or predating that lookup). Migration 003 backfill (repro7.py) yields content_items=1 with dedup_key='tg:fp:1000:60:480:640' and matched=0 -> the matched=1 row is dropped and its matched signal is LOST. Any downstream that keys off content_items.matched now sees the surviving row as unmatched.

evidence: src/tg_video_filter/db.py:537-568 (INSERT OR IGNORE ... UNIQUE(account_id,dedup_key), fp CASE branch at 554-557 omits source_id so it spans channels); git show main:db.py shows v1 has no fingerprint UNIQUE constraint (only the document_id partial index). Reproduced in scratchpad/repro7.py with a faithful v1 schema incl. idx_videos_user_document: 2 videos -> 1 content_item, matched signal LOST. Narrow trigger (needs coexisting fp-duplicate rows, which v1's ingest-time fingerprint lookup normally prevents), hence low.

recommendation: If the collapse is intended dedup, at minimum preserve the strongest state on conflict (e.g. backfill in matched DESC order, or after the INSERT run an UPDATE content_items SET matched=1 WHERE dedup_key IN (SELECT ... FROM videos_deprecated WHERE matched=1)), and/or add a post-migration row-count reconciliation so silent drops are visible.

2. assistant (2026-07-06T06:08:35.601Z)

I'll verify this finding by reading the actual code and tracing the control flow.

3. user (2026-07-06T06:08:36.783Z)

```

500     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
501     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
502     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
503     them), then backfills the three populated ones from the legacy
504     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
505     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506     pattern). Idempotent: a no-op once the legacy tables have been renamed.
507     """
508     # Idempotency gate: if the legacy videos table is gone, this migration has
509     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510     # dropped by _apply_schema post-003). Nothing to backfill.
511     if not await _table_exists(conn, "videos"):
512         return
513
514     # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515     # channels stay watched (watched=1). provider_source_id is the Telegram
516     # channel_id (cast to TEXT for the provider-neutral column).
517     await conn.execute(
518         """
519         INSERT OR IGNORE INTO sources
520             (account_id, provider, provider_source_id, title, kind, watched,
521              seen_first_at, last_seen_at)
522         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
523                1, seen_first_at, seen_first_at
524         FROM channels
525         """
526     )
527
528     # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
529     # The three-branch decision tree (Option 1A):
530     # a) document_id IS NOT NULL           ? 'tg:doc:' || document_id
531     # b) all four fingerprint fields set   ? 'tg:fp:' || size:dur:h:w
532     # c) else (Tier-0 fallback)           ? 'tg:msg:' || source_id:msg_id
533     # The ``CASE`` is ordered strongest-first and each branch is guarded so
534     # partial metadata never collapses to a degenerate key. source_id is
535     # resolved by joining the channel's provider_source_id back to sources.id

```

```

536 # (Telegram message_id is per-channel, so source_id is required).
537 await conn.execute(
538     """
539     INSERT OR IGNORE INTO content_items
540     (account_id, source_id, provider, provider_message_id, content_type,
541     document_id, duration_s, width, height, size_bytes, mime_type,
542     dedup_key, matched, seen_at)
543     SELECT
544     v.user_id,
545     s.id,
546     'telegram',
547     CAST(v.message_id AS TEXT),
548     'video',
549     CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
550     v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
551     CASE
552     WHEN v.document_id IS NOT NULL
553     THEN 'tg:doc:' || v.document_id
554     WHEN v.size_bytes IS NOT NULL AND v.duration_s IS NOT NULL
555     AND v.height IS NOT NULL AND v.width IS NOT NULL
556     THEN 'tg:fp:' || v.size_bytes || ':' || v.duration_s || ':'
557     || v.height || ':' || v.width
558     ELSE 'tg:msg:' || s.id || ':' || v.message_id
559     END,
560     v.matched,
561     v.seen_at
562     FROM videos v
563     JOIN sources s
564     ON s.account_id = v.user_id
565     AND s.provider = 'telegram'
566     AND s.provider_source_id = CAST(v.channel_id AS TEXT)
567     """
568 )
569
570 # 3. Backfill policies from filter_config: one 'default' policy per account
571 # (Option 2A). is_default=1 so the legacy load_filter_config shim reads
572 # this row. UNIQUE(account_id, name) dedups idempotent re-runs.
573 await conn.execute(
574     """
575     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
576     created_at, updated_at)
577     SELECT user_id, 'default', config_json, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
578     FROM filter_config
579     """
580 )
581
582 # 4. Rename (don't drop) the legacy tables ? kept for one release as a
583 # rollback safety net (ADR 0010 pattern).
584 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
586 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
587
588
589 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
590     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
591     async with conn.execute(
592         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
593     ) as cur:
594         return await cur.fetchone() is not None
595
596
597 # --- helpers -----
598
599

```

4. assistant (2026-07-06T06:08:39.223Z)

Now let me look at the schema for content_items to confirm the UNIQUE constraint, and check v1's schema.

5. user (2026-07-06T06:08:40.751Z)

```
5:for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
36: telegram_id INTEGER UNIQUE,
57: UNIQUE (user_id, channel_id)
74: UNIQUE (user_id, channel_id, message_id)
83:-- ``content_items``, and ``policies`` are populated by migration 003 (and by
98: UNIQUE (account_id, provider, provider_source_id)
101:CREATE TABLE IF NOT EXISTS content_items (
117: dedup_key TEXT NOT NULL,
120: UNIQUE (account_id, source_id, provider_message_id),
121: UNIQUE (account_id, dedup_key)
132: UNIQUE (account_id, name)
144: UNIQUE (account_id, provider, target)
156: UNIQUE (account_id, source_id, destination_id)
161: content_item_id INTEGER NOT NULL REFERENCES content_items(id),
170: UNIQUE (content_item_id, destination_id)
247:# Pure function: computes the ``content_items.dedup_key`` value from metadata
251:# See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
254:def compute_dedup_key(
265:     """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
282:     ``dedup_key`` is worse than a loud failure at ingestion time.
295:     "Telegram dedup_key fallback requires source_id and provider_message_id "
302:     f"dedup_key for provider={provider!r} requires source_id and
provider_message_id"
339:     "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
376: telegram_id INTEGER UNIQUE,
435: UNIQUE (user_id, channel_id)
457: UNIQUE (user_id, channel_id, message_id)
485: "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
500: Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
528: # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
539: INSERT OR IGNORE INTO content_items
542: dedup_key, matched, seen_at)
572: # this row. UNIQUE(account_id, name) dedups idempotent re-runs.
709: dedup_key=row["dedup_key"],
956: The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
958: - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
959: - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
962: """INSERT INTO content_items
965: size_bytes, mime_type, dedup_key, matched, seen_at)
982: item.dedup_key,
1000: """SELECT * FROM content_items
```

[Showing results with pagination = limit: 40]

6. assistant (2026-07-06T06:08:46.902Z)

The finding's core claims are confirmed by reading:

- Line 539: `INSERT OR IGNORE INTO content\_items`
- Line 121: `UNIQUE (account\_id, dedup\_key)`
- Lines 554-557: fp branch spans channels (no source_id in key)

Now the key question for the finding: can two videos with identical fingerprint, NULL document_id, and different matched values coexist in a v1 DB? Let me check v1's schema and the ingest-time fingerprint lookup.

7. user (2026-07-06T06:08:49.137Z)

```

28:CREATE TABLE IF NOT EXISTS accounts (
30:    telegram_id INTEGER UNIQUE,
36:CREATE TABLE IF NOT EXISTS telegram_sessions (
45:CREATE TABLE IF NOT EXISTS channels (
51:    UNIQUE (user_id, channel_id)
54:CREATE TABLE IF NOT EXISTS videos (
59:    document_id INTEGER,
68:    UNIQUE (user_id, channel_id, message_id)
71:CREATE TABLE IF NOT EXISTS filter_config (
77:# Note: the ``idx_videos_user_document`` partial unique index is created by
78:# migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
159:async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
160:    """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
162:    Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
165:    if not await _column_exists(conn, "videos", "document_id"):
166:        await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
167:    # Create the partial unique index for both fresh and migrated DBs. Uses
170:    "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
171:    "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
205:    CREATE TABLE accounts (
207:        telegram_id INTEGER UNIQUE,
226:    CREATE TABLE telegram_sessions (
260:    CREATE TABLE channels (
266:        UNIQUE (user_id, channel_id)
274:    CREATE TABLE videos (
279:        document_id INTEGER,
288:        UNIQUE (user_id, channel_id, message_id)
291:        "id, user_id, channel_id, message_id, document_id, duration_s, width, "
297:    CREATE TABLE filter_config (
312:    # The partial unique index for document_id dedup (created by migration 001
316:    "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
317:    "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
399:    document_id=row["document_id"],
622:async def get_video_by_document_id(
623:    conn: aiosqlite.Connection, user_id: int, document_id: int
633:    WHERE user_id = ? AND document_id = ?""",
634:    (user_id, document_id),
640:async def get_video_by_content_fingerprint(
648:    """Tier-2 content dedup: metadata fingerprint match ? probably same video.
674:    - ``UNIQUE (user_id, channel_id, message_id)`` ? same message instance.
675:    - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
680:    (user_id, channel_id, message_id, document_id, duration_s, width, height,
687:    video.document_id,
765:    "get_video_by_document_id",
766:    "get_video_by_content_fingerprint",

```

8. assistant (2026-07-06T06:08:53.414Z)

Confirmed: v1 has NO DB-level fingerprint uniqueness. Only `UNIQUE(user\_id, channel\_id, message\_id)` and the partial `idx\_videos\_user\_document`. So two fp-duplicate rows with NULL document_id in different channels CAN coexist. Now let me verify the collapse actually drops the matched=1 row via a repro.

9. user (2026-07-06T06:08:54.673Z)

```

300    if source_id is None or provider_message_id is None:
301        raise ValueError(
302            f"dedup_key for provider={provider!r} requires source_id and
provider_message_id"
303        )
304    return f"{p}:msg:{source_id}:{provider_message_id}"
305
306
307    async def _apply_migrations(conn: aiosqlite.Connection) -> None:
308        """Run every registered migration whose version > current ``user_version``."""

```

```

309         async with conn.execute("PRAGMA user_version") as cur:
310             row = await cur.fetchone()
311             current = int(row[0]) if row else 0
312
313         for version, fn in _MIGRATIONS:
314             if version <= current:
315                 continue
316             await fn(conn)
317             await conn.execute(f"PRAGMA user_version = {version}")
318
319
320     async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
321         """Return True if ``column`` exists on ``table`` (async, PRAGMA table_info)."""
322         async with conn.execute(f"PRAGMA table_info({table})") as cur:
323             rows = await cur.fetchall()
324             return any(row[1] == column for row in rows)
325
326
327     @_migration(1)
328     async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
329         """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
330
331         Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
332         created before this migration existed.
333         """
334         if not await _column_exists(conn, "videos", "document_id"):
335             await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
336         # Create the partial unique index for both fresh and migrated DBs. Uses
337         # IF NOT EXISTS so it is safe on DBs where it was already created.
338         await conn.execute(
339             "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
340             "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
341         )
342
343
344     @_migration(2)
345     async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
346         """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
347
348         ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
349         data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,

```

10. assistant (2026-07-06T06:08:57.653Z)

Now let me build a repro. I'll construct a v1-shape DB directly (videos table with two fp-duplicate rows, different channels, different matched), then run the migration 003 backfill.

11. user (2026-07-06T06:11:16.464Z)

Command running in background with ID: b6fl5ahxe. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b6fl5ahxe.output. You will be notified when it completes. To check interim output, use Read on that file path.

12. assistant (2026-07-06T06:11:19.555Z)

Let me find the migration function name and check output.

13. user (2026-07-06T06:11:24.284Z)

```

496:@_migration(3)
497:async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
498:    """
499:    """
500:
501:    """
502:    """
503:
504:    """
505:    """
506:
507:    """
508:    """
509:
510:    """
511:    """
512:
513:    """
514:    """
515:
516:    """
517:    """
518:
519:    """
520:    """
521:
522:    """
523:    """
524:
525:    """
526:    """
527:
528:    """
529:    """
530:
531:    """
532:    """
533:
534:    """
535:    """
536:
537:    """
538:    """
539:
540:    """
541:    """
542:
543:    """
544:    """
545:
546:    """
547:    """
548:
549:    """
550:    """
551:
552:    """
553:    """
554:
555:    """
556:    """
557:
558:    """
559:    """
560:
561:    """
562:    """
563:
564:    """
565:    """
566:
567:    """
568:    """
569:
570:    """
571:    """
572:
573:    """
574:    """
575:
576:    """
577:    """
578:
579:    """
580:    """
581:
582:    """
583:    """
584:
585:    """
586:    """
587:
588:    """
589:    """
590:
591:    """
592:    """
593:
594:    """
595:    """
596:
597:    """
598:    """
599:
600:    """
601:    """
602:
603:    """
604:    """
605:
606:    """
607:    """
608:
609:    """
610:    """
611:
612:    """
613:    """
614:
615:    """
616:    """
617:
618:    """
619:    """
620:
621:    """
622:    """
623:
624:    """
625:    """
626:
627:    """
628:    """
629:
630:    """
631:    """
632:
633:    """
634:    """
635:
636:    """
637:    """
638:
639:    """
640:    """
641:
642:    """
643:    """
644:
645:    """
646:    """
647:
648:    """
649:    """
650:
651:    """
652:    """
653:
654:    """
655:    """
656:
657:    """
658:    """
659:
660:    """
661:    """
662:
663:    """
664:    """
665:
666:    """
667:    """
668:
669:    """
670:    """
671:
672:    """
673:    """
674:
675:    """
676:    """
677:
678:    """
679:    """
680:
681:    """
682:    """
683:
684:    """
685:    """
686:
687:    """
688:    """
689:
690:    """
691:    """
692:
693:    """
694:    """
695:
696:    """
697:    """
698:
699:    """
700:    """
701:
702:    """
703:    """
704:
705:    """
706:    """
707:
708:    """
709:    """
710:
711:    """
712:    """
713:
714:    """
715:    """
716:
717:    """
718:    """
719:
720:    """
721:    """
722:
723:    """
724:    """
725:
726:    """
727:    """
728:
729:    """
730:    """
731:
732:    """
733:    """
734:
735:    """
736:    """
737:
738:    """
739:    """
740:
741:    """
742:    """
743:
744:    """
745:    """
746:
747:    """
748:    """
749:
750:    """
751:    """
752:
753:    """
754:    """
755:
756:    """
757:    """
758:
759:    """
760:    """
761:
762:    """
763:    """
764:
765:    """
766:    """
767:
768:    """
769:    """
770:
771:    """
772:    """
773:
774:    """
775:    """
776:
777:    """
778:    """
779:
780:    """
781:    """
782:
783:    """
784:    """
785:
786:    """
787:    """
788:
789:    """
790:    """
791:
792:    """
793:    """
794:
795:    """
796:    """
797:
798:    """
799:    """
800:
801:    """
802:    """
803:
804:    """
805:    """
806:
807:    """
808:    """
809:
810:    """
811:    """
812:
813:    """
814:    """
815:
816:    """
817:    """
818:
819:    """
820:    """
821:
822:    """
823:    """
824:
825:    """
826:    """
827:
828:    """
829:    """
830:
831:    """
832:    """
833:
834:    """
835:    """
836:
837:    """
838:    """
839:
840:    """
841:    """
842:
843:    """
844:    """
845:
846:    """
847:    """
848:
849:    """
850:    """
851:
852:    """
853:    """
854:
855:    """
856:    """
857:
858:    """
859:    """
860:
861:    """
862:    """
863:
864:    """
865:    """
866:
867:    """
868:    """
869:
870:    """
871:    """
872:
873:    """
874:    """
875:
876:    """
877:    """
878:
879:    """
880:    """
881:
882:    """
883:    """
884:
885:    """
886:    """
887:
888:    """
889:    """
890:
891:    """
892:    """
893:
894:    """
895:    """
896:
897:    """
898:    """
899:
900:    """
901:    """
902:
903:    """
904:    """
905:
906:    """
907:    """
908:
909:    """
910:    """
911:
912:    """
913:    """
914:
915:    """
916:    """
917:
918:    """
919:    """
920:
921:    """
922:    """
923:
924:    """
925:    """
926:
927:    """
928:    """
929:
930:    """
931:    """
932:
933:    """
934:    """
935:
936:    """
937:    """
938:
939:    """
940:    """
941:
942:    """
943:    """
944:
945:    """
946:    """
947:
948:    """
949:    """
950:
951:    """
952:    """
953:
954:    """
955:    """
956:
957:    """
958:    """
959:
960:    """
961:    """
962:
963:    """
964:    """
965:
966:    """
967:    """
968:
969:    """
970:    """
971:
972:    """
973:    """
974:
975:    """
976:    """
977:
978:    """
979:    """
980:
981:    """
982:    """
983:
984:    """
985:    """
986:
987:    """
988:    """
989:
990:    """
991:    """
992:
993:    """
994:    """
995:
996:    """
997:    """
998:
999:    """
1000:    """

```

Traceback (most recent call last):

```

File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_verify.py", line 59, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~

File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_verify.py", line 34, in main
    await conn.execute("INSERT INTO accounts (id, telegram_id) VALUES (1, 111)")
    ~~~~~^~~~~~

File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223, in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
    ~~~~~^~~~~~

File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160, in _execute
    return await future
    ~~~~~^~~~~~

File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in _connection_worker_thread
    result = function()
    ~~~~~^~~~~~

sqlite3.IntegrityError: NOT NULL constraint failed: accounts.created_at

```

Schema differences. Let me check the accounts table columns and adjust.

```

25         Source,
26         TelegramSession,
27         User,
28         VideoRecord,
29     )
30
31 # --- DDL -----
32
33 _SCHEMA = ""
34 CREATE TABLE IF NOT EXISTS accounts (
35     id            INTEGER PRIMARY KEY,
36     telegram_id   INTEGER UNIQUE,
37     provider      TEXT NOT NULL DEFAULT 'telegram',
38     display_name  TEXT,
39     created_at    TEXT NOT NULL
40 );
41
42 CREATE TABLE IF NOT EXISTS telegram_sessions (
43     account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
44     phone         TEXT,
45     session_path  TEXT NOT NULL,
46     enabled       INTEGER NOT NULL DEFAULT 1,
47     created_at    TEXT NOT NULL,
48     last_login_at TEXT
49 );
50
51 CREATE TABLE IF NOT EXISTS channels (

```

```

52         id            INTEGER PRIMARY KEY AUTOINCREMENT,
53         user_id        INTEGER NOT NULL REFERENCES accounts(id),
54         channel_id     INTEGER NOT NULL,
55         title          TEXT,
56         seen_first_at  TEXT NOT NULL,
57         UNIQUE (user_id, channel_id)
58     );
59
60     CREATE TABLE IF NOT EXISTS videos (
61         id            INTEGER PRIMARY KEY AUTOINCREMENT,
62         user_id        INTEGER NOT NULL REFERENCES accounts(id),
63         channel_id     INTEGER NOT NULL,
64         message_id     INTEGER NOT NULL,
65         document_id    INTEGER,
66         duration_s     INTEGER,
67         width          INTEGER,
68         height         INTEGER,
69         size_bytes     INTEGER,
70         mime_type      TEXT,
71         matched        INTEGER NOT NULL DEFAULT 0,
72         forwarded      INTEGER NOT NULL DEFAULT 0,
73         seen_at        TEXT NOT NULL,
74         UNIQUE (user_id, channel_id, message_id)
75     );
76
77     CREATE TABLE IF NOT EXISTS filter_config (
78         user_id        INTEGER PRIMARY KEY REFERENCES accounts(id),
79         config_json    TEXT NOT NULL
80     );
81
82     -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
83     -- ``content_items``, and ``policies`` are populated by migration 003 (and by
84     -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
85     -- are created empty here so the schema is complete ? Track C (#28) implements
86     -- the delivery logic that populates them.
87
88     CREATE TABLE IF NOT EXISTS sources (
89         id            INTEGER PRIMARY KEY AUTOINCREMENT,
90         account_id     INTEGER NOT NULL REFERENCES accounts(id),
91         provider       TEXT NOT NULL DEFAULT 'telegram',
92         provider_source_id TEXT NOT NULL,
93         title          TEXT,
94         kind           TEXT NOT NULL DEFAULT 'channel',
95         watched        INTEGER NOT NULL DEFAULT 0,
96         seen_first_at  TEXT NOT NULL,
97         last_seen_at   TEXT,
98         UNIQUE (account_id, provider, provider_source_id)
99     );

```

16. assistant (2026-07-06T06:11:32.121Z)

The `\_apply\_schema` already creates the `videos`/`channels`/`filter\_config` legacy tables with full columns (they use `IF NOT EXISTS`). I don't need to recreate them ? I'll just use them directly. Let me simplify the repro.

17. user (2026-07-06T06:13:46.639Z)

Command running in background with ID: bn6ryg2wu. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bn6ryg2wu.output. You will be notified when it completes. To check interim output, use Read on that file path.

18. user (2026-07-06T06:13:55.059Z)


```

videos exists: False
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_verify2.py", line 40, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro_verify2.py", line 15, in main
    await conn.execute("INSERT INTO channels (user_id, channel_id, title, seen_first_at) VALUES
(1, 100, 'chanA', '2020-01-01')")
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223, in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160, in _execute
    return await future
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in _connection_worker_thread
    result = function()
sqlite3.OperationalError: no such table: channels

```

19. assistant (2026-07-06T06:13:59.484Z)

`\_apply\_schema` drops the legacy tables (videos/channels/filter_config) after running migration 003. So to repro I must build the legacy tables myself. Let me check what `\_apply\_schema` does at the end.

20. user (2026-07-06T06:14:01.181Z)

```

191:async def _apply_schema(conn: aiosqlite.Connection) -> None:
201:    # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
204:    if await _table_exists(conn, "channels_deprecated"):
205:        await conn.execute("DROP TABLE IF EXISTS channels")
206:        await conn.execute("DROP TABLE IF EXISTS videos")
207:        await conn.execute("DROP TABLE IF EXISTS filter_config")
352:    ``users_deprecated`` and kept for one release as a rollback safety net; a
358:    those FKs to point at ``users_deprecated``, diverging from fresh installs
371:    await conn.execute("DROP TABLE IF EXISTS accounts")
392:    await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
420:    # the rename ? so a naive rename of users ? users_deprecated would leave
421:    # these tables referencing users_deprecated(id), diverging from fresh DBs
479:    await conn.execute(f"DROP TABLE {table}__legacy_data")
493:    await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
505:    legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
584:    await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585:    await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
586:    await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
1082:    ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1281:    ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``

```

21. user (2026-07-06T06:14:05.900Z)

```
191     async def _apply_schema(conn: aiosqlite.Connection) -> None:
192         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
193         # and a busy timeout so concurrent access waits instead of failing immediately.
194         await conn.execute("PRAGMA journal_mode = WAL")
195         await conn.execute("PRAGMA foreign_keys = ON")
196         await conn.execute("PRAGMA busy_timeout = 5000")
197         await conn.executescript(_SCHEMA)
198         # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
199         # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
200         # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
201         # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
202         # any *empty* recreations of those legacy tables from a subsequent
203         # ``init_db`` are dropped here so they don't shadow the renamed copies.
204         if await _table_exists(conn, "channels_deprecated"):
205             await conn.execute("DROP TABLE IF EXISTS channels")
206             await conn.execute("DROP TABLE IF EXISTS videos")
207             await conn.execute("DROP TABLE IF EXISTS filter_config")
208         await _apply_migrations(conn)
209         await conn.commit()
210
211
212     # --- migrations -----
213     #
214     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
215     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
216     # the source of truth for *fresh* databases; the migrations below reconcile
217     # *existing* databases that were created by an older schema version.
218     #
219     # To add a new migration:
220     #   1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
```

22. assistant (2026-07-06T06:14:10.805Z)

Now I see. `\_apply\_schema` calls `\_apply\_migrations` at the end (line 208), which runs migration 003 automatically since user_version starts at 0. So my repro's `\_apply\_schema` already ran 003 (dropping videos). I need to seed the legacy `videos` BEFORE migration 003 runs. Let me build the legacy schema manually then call migration 003 directly.

23. user (2026-07-06T06:14:27.623Z)

```
videos exists: True
content_items count: 1
{'id': 1, 'source_id': 1, 'provider_message_id': '500', 'dedup_key': 'tg:fp:1000:60:480:640',
 'matched': 0}
legacy matched=1 rows: 1
SURVIVING matched signal lost? True
```

24. assistant (2026-07-06T06:14:32.562Z)

Reproduced. The two fp-duplicate videos (matched=0 at rowid 1, matched=1 at rowid 2) collapse to a single `content\_items` row with `matched=0` and `dedup\_key='tg:fp:1000:60:480:640'`. The `matched=1` signal is dropped ? INSERT OR IGNORE keeps the lower-rowid (matched=0) row.

Now the adversarial question: ****can this precondition actually occur in a real v1 DB?**** The finding rests on two fp-duplicate rows with NULL document_id coexisting. Let me check the ingest-time fingerprint lookup path referenced (telethon_client.py:207-222) to see whether it would normally prevent this.

25. user (2026-07-06T06:14:33.549Z)

```
195         log.info(
196             "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
```

```

197         "original_channel=%s original_msg=%s",
198         user.id,
199         document_id,
200         channel_id,
201         message_id,
202         existing.channel_id,
203         existing.message_id,
204     )
205     return
206
207     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
208     if (
209         meta.size_bytes is not None
210         and meta.duration_s is not None
211         and meta.height is not None
212         and meta.width is not None
213     ):
214         existing = await db.get_video_by_content_fingerprint(
215             conn,
216             user.id,
217             meta.size_bytes,
218             meta.duration_s,
219             meta.height,
220             meta.width,
221         )
222         if existing is not None:
223             log.info(
224                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
225                 "channel=%s msg=%s original_channel=%s original_msg=%s",
226                 user.id,
227                 meta.size_bytes,
228                 meta.duration_s,
229                 meta.height,
230                 meta.width,
231                 channel_id,
232                 message_id,
233                 existing.channel_id,
234                 existing.message_id,
235             )
236         return
237
238     # Lazily record the channel (best-effort; failures are non-fatal).
239     try:

```

26. assistant (2026-07-06T06:14:46.088Z)

So v1 ingest DOES have a Tier-2 fingerprint lookup that returns early (skips insert) when a fingerprint match is found. This means in normal operation, the second fp-duplicate would NOT be inserted. This narrows the trigger significantly. But the finding explicitly acknowledges this and rates it **low** for that reason.

The precondition can still occur because:

1. This Tier-2 lookup is application-level, not DB-enforced ? a race between two concurrent ingests (both check, both find nothing, both insert) can produce coexisting fp-duplicates.
2. Videos predating the fingerprint-lookup feature (it's a later ADR) could coexist.

Both are real but narrow. Let me confirm the fingerprint lookup is guarded only at app level (no DB constraint) ? already confirmed from v1 schema grep: only ``UNIQUE(user_id,channel_id,message_id)`` and partial ``idx_videos_user_document``. No fingerprint UNIQUE. So the DB permits fp-duplicates; the app-level check is best-effort and racy.

The finding is accurate: real defect, produces the described wrong behavior (matched=1 silently dropped), ships in this PR's diff (lines 537-568, new code), narrow trigger. Severity low is appropriate ? the trigger requires coexisting fp-duplicate rows which the app-level Tier-2 lookup normally prevents, so it's an edge case, but when it happens the matched signal is

genuinely lost with no reconciliation.

One more check: is the loss actually consequential? `matched` drives downstream behavior. But even accepting it matters, the low rating already reflects the realistic likelihood. The claim is confirmed.

27. user (2026-07-06T06:14:56.237Z)

Structured output provided successfully